

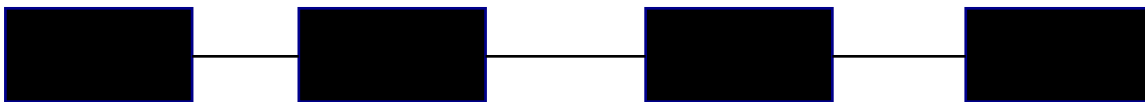
Federated Authentication Guide

Social Identity Providers, AWS Cognito, OAuth 2.0, OIDC, and Enterprise SSO

Overview

Federated authentication allows applications to delegate user authentication to trusted identity providers. AWS Cognito commonly acts as an identity broker between applications and external providers such as Google, GitHub, LinkedIn, Azure AD (Microsoft Entra ID), or enterprise identity systems.

Social Login Flow



How Social Login Works

1. User clicks Sign in with Google/GitHub/LinkedIn.
2. Cognito redirects to the provider.
3. The provider authenticates the user.
4. Cognito validates the identity and creates/maps the user in the User Pool.
5. Cognito issues JWT tokens to the application.

OAuth 2.0 and OpenID Connect (OIDC)

OAuth 2.0 provides authorization and access tokens. **OIDC** extends OAuth 2.0 by adding authentication through an ID Token containing user identity claims.

Example Social Login JWT

```
{
  "email": "john@gmail.com",
  "cognito:username": "Google_123456",
  "identities": [{"providerName": "Google", "userId": "123456"}]
}
```

Key Claims

- `identities[].providerName` identifies the source provider.
- `cognito:username` is generated by Cognito and often prefixed with the provider name.

Enterprise Federation: Cognizant SSO

Enterprise SSO Flow



In enterprise federation, employees authenticate using corporate credentials managed by Azure AD (Microsoft Entra ID) or another enterprise identity provider. Cognito trusts the enterprise provider and issues application tokens after successful authentication.

Example Enterprise JWT

```
{
  "email": "employee@cognizant.com",
  "cognito:username": "AzureAD_abcdef",
  "identities": [{"providerName": "AzureAD"}]
}
```

Benefits of Cognito as an Identity Broker

- Single integration point for multiple identity providers.
- Centralized user management.
- Consistent JWT token structure.
- Easier role mapping and authorization.
- Supports both social and enterprise authentication.